

Week 11: Bayesian approaches*

Emma James

1 Introduction

Learning objectives

As a reminder, the learning objectives for this week are:

1. Recall core principles of frequentist statistics
2. Describe the philosophy behind Bayesian statistics
3. Explain how prior and posterior probability distributions are used in Bayesian inference
4. Compare alternative models by interpreting Bayes Factors as evidence
5. Calculate and interpret a simple Bayes Factor in R

In this final practical, we'll see how we can calculate and interpret Bayes Factors for simple analyses using the *BayesFactor* package (Morey & Rouder, 2024). This can be an effective way of incorporating Bayesian methods alongside standard frequentist approaches for hypothesis testing. Part A will give you just one example of using an analysis function from the *BayesFactor* package, before we get stuck in with some more interesting data from a “digital detox” trial in adolescents.

2 Setting up

2.1 Packages

Today's practical will use the following packages. You will need to install each one before loading them at the top of your script.

```
library(tidyverse)
library(naniar)
library(BayesFactor)
```

You might also find it helpful to turn off scientific notation using `options(scipen=999)`.

*These materials are adapted from an original version by Dan Baker.

3 Part A: Core example

3.1 Introduction to the dataset

Let's start with a quick demonstration using a built-in dataset as usual. For this purpose, we can use the `penguins` dataset (admittedly it's not "human data"—we'll do that next—but who doesn't like penguins?!). The dataset contains measurements from three species of penguin collected from three islands in the Antarctica.

The *BayesFactor* package doesn't play nicely with missing data, so we will also filter the data for complete cases. Of course if this was a real analysis we would do this much more cautiously.

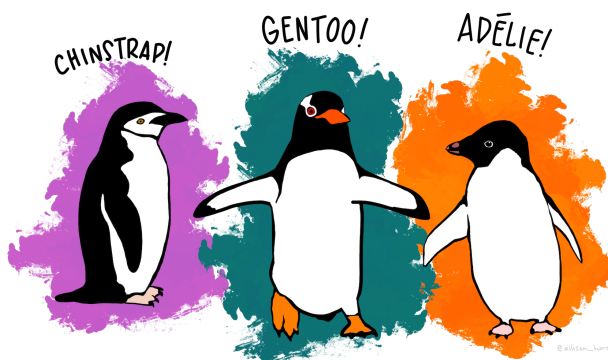


Figure 1: Meet the penguins! Artwork by @allison_horst

```
# Load dataset into the environment
data(penguins)

# Filter for complete cases
penguins_cc <- penguins %>%
  filter(complete.cases(.))
```

Use R code to answer the following questions about the dataset:

- How many observations did we lose by listwise deletion?
- How many remaining penguins of each species are there?
- How many penguins of each species were on each island?
- Which island had the largest penguins in terms of average body mass?
- Where was the penguin with the shortest bill length (`bill_len`) found, and what species was it?
- What is the average body mass and bill length of male versus female penguins?

```

# Listwise deletion
nrow(penguins) - nrow(penguins_cc)

# Species count
penguins_cc %>%
  group_by(species) %>%
  count()

# Species by island
penguins_cc %>%
  group_by(island, species) %>%
  count()

# Size by island
penguins_cc %>%
  group_by(island) %>%
  summarise(mean = mean(body_mass))

# Shortest bill penguin
penguins_cc %>%
  slice_min(bill_len)

# Sex differences
penguins_cc %>%
  group_by(sex) %>%
  summarise(mass = mean(body_mass),
            bill_length = mean(bill_len))

```

3.2 Independent t-test

For our example, we will use a simple independent t-test to compare the bill length of male versus female penguins. We can first remind ourselves how to run a frequentist independent t-test:

3.2.1 Frequentist

```
t.test(bill_len ~ sex, data = penguins_cc)
```

You should see that there is a significant difference between groups, with male penguins averaging ~3mm longer beaks than female penguins.

3.2.2 Bayes Factor

Now let's calculate a Bayes Factor for this test. We use the specialised t-test function from the *BayesFactor* package, but everything else is set up in pretty much the same way. The only difference is that we must

explicitly specify that the first argument we provide is a formula.

```
ttestBF(formula = bill_len ~ sex, data = penguins_cc)
```

The output tells us the direction of the comparison—i.e., that it is testing evidence for the alternative hypothesis against evidence for the null hypothesis that the difference in means is 0 (BF_{10}). The alternative hypothesis used the default Cauchy prior, which you can see in the output is scaled to “ $r = 0.707$ ”. If you’re interested in the justification for this decision, the references are linked in the “Note” of the function documentation (i.e., call `help(ttestBF)`). We’ll explore the implications of using different priors below.

The output shows a *very* large BF in favour of the alternative hypothesis (>73 million!), which is perhaps not surprising considering the clear group difference observed above.

For comparison, let’s compare the bill length of penguins from two different islands. Filter the data as follows, and run a Bayes Factor t-test:

```
# Remove penguins from Torgersen island
penguins_2isl <- filter(penguins_cc, island != "Torgersen") %>%
  droplevels()

ttestBF(formula = bill_len ~ island, data = penguins_2isl)
```

This time our Bayes Factor is < 1 , indicating evidence in favour of the null hypothesis. However, we can consider it weak or “anecdotal” evidence with reference to Jeffreys’ scale as it is larger than $1/3$.

Some people find it easier to flip the Bayes Factor to report evidence in favour of the null over the alternative hypothesis (BF_{01} ; values >1 favour the null). We can do this simply by inverting the test:

```
1/ttestBF(formula = bill_len ~ island, data = penguins_2isl)
```

Congratulations, you’ve just calculated your first Bayes Factors! It was surprisingly easy, huh?

3.3 Changing the prior

The *BayesFactor* package uses a default, weakly informative prior to test hypotheses—meaning that we didn’t actually use much in the way of specialist knowledge when calculating the Bayes Factors above. The default is to use a [Cauchy distribution](#) (a bit like a pointier normal distribution) with a scale of 0.707. Plot this to see what it looks like, using the following line of code:

```
# Set scale of x axis
x <- seq(-10, 10, by = 0.1)
plot(x, dcauchy(x, scale = 0.707), type = "l")
```

Using a distribution (rather than a single point estimate) allows us to consider that a range of outcomes are plausible. It is centered around 0 to indicate that the effect may be in either direction, and has more mass over small versus large values as very large effect sizes tend to be unlikely.

See what happens to the distribution when you change the scale value from lower to higher numbers. How does this affect the width of the prior?

Now let's see how changing the width of the prior affects the Bayes Factor, using our original t-test example. Try a range of smaller and higher values for the `rscale` argument. How does the prior affect the conclusions?

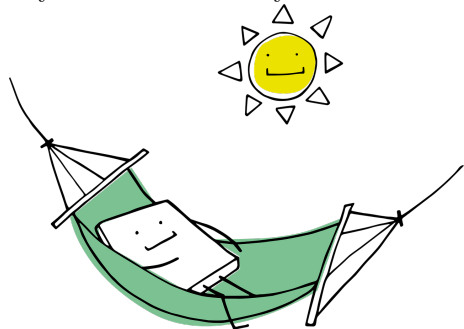
```
# Default
ttestBF(formula = bill_len ~ island, data = penguins_2isl)

# Alternatives - e.g.,
ttestBF(formula = bill_len ~ island, data = penguins_2isl, rscale = 0.3)
ttestBF(formula = bill_len ~ island, data = penguins_2isl, rscale = 3)
```

Researchers will often test a range of priors in a *sensitivity analysis*, allowing them to consider how the choice of prior influences the results. We'll have a go at this in Part C.

💡 Take a break!

If you haven't already taken a break today, we suggest you do so before you start the next section.



4 Part B: Apply your knowledge

Let's move onto some real data. Over the module, you've built skills to explore and work with different packages in R. Now you've seen what the *BayesFactor* package is and what it does, you can use the [package help page](#) to identify the relevant functions for this section.

4.1 Introduction to the dataset

Today's dataset is from a feasibility trial conducted by Professor Lisa Henderson and Dr Emma Sullivan. They assessed the impact of a 3-week smartphone and social media ban on adolescents' sleep, cognition, and wellbeing. The trial was part of a TV documentary, *Swiped: the school that banned smartphones* ([Channel 4](#); [Netflix](#)). The results were released as a [pre-print](#), and will now form the basis of a [large-scale randomised control trial](#) here in York.

We will work with two types of data so that we can test out different analyses: the questionnaire data from before and after the ban, and data from Garmin watches that recorded sleep duration over the month period. You can copy the code below to create the key variables we'll need (summed anxiety score, averaged sleep data), though you may need to do some additional data wrangling for some of the analyses below.

```
# Load the questionnaire data
swiped_q_dat <- read_csv("https://osf.io/cpz2n/?action=download") %>%
  filter(Condition == 1) %>% # Pupils who took part in ban
  select(random_ID, Session, PSQI_Q4, contains("HADS_")) %>%
  replace_with_na_at(.vars = 5:12, condition = ~.x == 5) %>% # code missing data
  mutate(HADS_anx = HADS_Q1 + HADS_Q3 + HADS_Q5 +      # Sum anxiety items
          HADS_Q7 + HADS_Q9 + HADS_Q11 + HADS_Q13) %>%
  select(-contains("HADS_Q")) %>%
  pivot_wider(names_from = "Session", values_from = c(PSQI_Q4, HADS_anx)) %>%
  mutate_at(vars(random_ID), as.factor)

# Load the wearables data
swiped_w_dat <- read_csv("https://osf.io/k62u5/?action=download") %>%
  filter(Weekday == 1) %>% # Use sleep duration on school nights
  select(random_ID, Ban, Day, Sleep_duration) %>%
  group_by(random_ID, Ban) %>%
  summarise(Sleep_duration = mean(Sleep_duration, na.rm = TRUE)) %>%
  mutate_at(vars(random_ID, Ban), as.factor)
```

The questionnaire data (`swiped_q_dat`) should have the following variables remaining:

- **random_ID**: a unique participant code
- **PSQI_Q4_***: self-reported sleep duration in hours; before (1) and after (2) the ban
- **HADS_anx_***: anxiety scale score from the Hospital Depression and Anxiety Scale; before (1) and after (2) the ban

The wearables data (`swiped_w_dat`) should have the following variables remaining:

- **random_ID**: a unique participant code
- **Ban**: ban period (1 = pre-ban; 2 = one week post-ban; 3 = two weeks post-ban; 4 = three weeks post-ban)
- **Sleep_duration**: mean sleep duration per night (hours) recorded during the ban period

Use R code to answer the following questions about the dataset:

- How many pupils took part in the phone ban? (*Hint: we might not be left with both types of data for all participants!*)
- What was the mean and range of *self-reported* sleep duration before and after the study?
- What was the mean and range of *recorded* sleep duration in before (Ban == 1) and at the end (Ban == 4) of the study?

```
# Number of pupils - can explore in various ways, e.g.,
length(unique(c(swiped_w_dat$random_ID, swiped_q_dat$random_ID)))

# Self-reported sleep duration
mean(swiped_q_dat$PSQI_Q4_1)
mean(swiped_q_dat$PSQI_Q4_2)
range(swiped_q_dat$PSQI_Q4_1)
range(swiped_q_dat$PSQI_Q4_2)

# Measured sleep duration
swiped_w_dat %>%
  filter(Ban == 1 | Ban == 4) %>%
  group_by(Ban) %>%
  summarise(mean = mean(Sleep_duration),
            min = min(Sleep_duration),
            max = max(Sleep_duration))
```

4.2 Analysing the data

Run both standard frequentist and Bayesian analyses to address the following questions. For each one, use the [package page](#) to identify the relevant function and inspect the examples to set up your model.

- Use a paired t-test to assess whether self-reported sleep duration and anxiety symptoms changed from before to after the ban (*hint: you will need to calculate difference scores*)
- Use a one-way ANOVA to assess whether the watch-recorded sleep duration changed across the four ban periods
- Use a regression model to assess whether change in self-reported sleep duration predicts change in anxiety symptoms

For each analysis, consider what the Bayes Factor adds to your understanding of the available evidence.

```
# Sleep duration - frequentist (usual format)
t.test(swiped_q_dat$PSQI_Q4_1, swiped_q_dat$PSQI_Q4_2, paired = TRUE)

# Create difference score and remove NAs
swiped_q_sleep <- swiped_q_dat %>%
  mutate(sleep_diff = PSQI_Q4_2-PSQI_Q4_1) %>%
  filter(!is.na(sleep_diff))

# Reformatted frequentist with difference score - same results
```

```

t.test(swiped_q_sleep$sleep_diff)

# Sleep duration - BF10
ttestBF(swiped_q_sleep$sleep_diff)

#####

# Anxiety - reformat and remove NAs
swiped_q_anx <- swiped_q_dat %>%
  mutate(anx_diff = HADS_anx_2-HADS_anx_1) %>%
  filter(!is.na(anx_diff))

# Anxiety - frequentist
t.test(swiped_q_anx$anx_diff)

# Anxiety - Bayes
ttestBF(swiped_q_anx$anx_diff) # BF10
1/ttestBF(swiped_q_anx$anx_diff) # or BF01

# Run and print ANOVA output
sleep_phase_mod <- aov(Sleep_duration ~ Ban + Error(random_ID/Ban), data = swiped_w_dat)
summary(sleep_phase_mod)

# Run and print Bayes Factor ANOVA
anovaBF(Sleep_duration ~ Ban + random_ID, whichRandom = "random_ID", data = swiped_w_dat) #BF10
1/anovaBF(Sleep_duration ~ Ban + random_ID, whichRandom = "random_ID", data = swiped_w_dat) #BF01

# Reformatting
swiped_q_dat <- swiped_q_dat %>%
  mutate(sleep_diff = PSQI_Q4_2-PSQI_Q4_1,
         anx_diff = HADS_anx_2-HADS_anx_1) %>%
  filter(!is.na(sleep_diff) & !is.na(anx_diff))

# Frequentist regression
sleep_anx_mod <- lm(sleep_diff ~ anx_diff, data = swiped_q_dat)
summary(sleep_anx_mod)

# Bayes factor regression
regressionBF(sleep_diff ~ anx_diff, data = swiped_q_dat) #BF10
1/regressionBF(sleep_diff ~ anx_diff, data = swiped_q_dat) #BF01

```

You should finish these in good time, in which case I highly recommend taking on today's challenge section...

5 Part C: Challenge yourself!

Run a full sensitivity analysis on your t-test for self-reported sleep duration before versus after the ban. The aim is to produce a plot that shows how the Bayes Factor changes as a function of different priors that you could have chosen. There is a beautiful illustration of this process by [Jim Grange on his blog](#).

You will need to create a loop (like you did in the stochastic methods practical) and extract the Bayes Factor values for each analysis. Here are some hints:

- *Before* your loop, you will need to create an empty list to store Bayes Factors.
- You might also want to create a row counter variable that starts at 0, and increases by one each time through the loop. You can use this variable to index which row/entry of your list to store the output in each time.
- You can use the `seq()` function to specify the different scale values. Try increasing the scale from 0.1 to 3, in steps of 0.1.
- You can extract the Bayes Factor from the model output as follows: `bf_list[n] <- extractBF(bf, onlybf = TRUE)`, where `bf` is the output of the `ttestBF()` command above, and `bf_list` is the name of your list to store the output.
- You can plot your different scale values on the x-axis, and resulting Bayes Factors on the y-axis.

Is the function a straight line, or does it peak around some optimal value? Would your conclusions about these data have changed based on the values of the prior that you selected?

```
# Create empty list to store BFs
bf_list <- NULL

# Create row counter
n <- 0

# Loop through tests
for (r in seq(0.1, 3, 0.1)){

  # Update row counter
  n <- n+1

  # Compute Bayes Factor with new scale value
  bf <- ttestBF(swiped_q_dat$sleep_diff, rscale = r)  # BF10

  # Extract BF and store in list
  bf_list[n] <- extractBF(bf, onlybf = TRUE)
}

# Plot
plot(seq(0.1, 3, 0.1), bf_list, type = "l")
```

6 Summary

6.1 Recap

This week served to remind us what we can and cannot conclude from null hypothesis significance testing. Bayes Factors can prove a useful tool in evaluating the strength of evidence for different hypotheses, and thereby allows us to consider whether a null effect is likely or inconclusive. The small-scale trial data was a good example of this in practice: the small sample had low statistical power for detecting significant effects under a frequentist framework, but the Bayes Factors allowed us to consider the likelihood that some outcomes were unaffected by the phone ban versus there not being sufficient evidence to draw conclusions. If you took on Part C, you'll have also seen how priors can affect the conclusions, and why sensitivity checks can play an important role in communicating results.

6.2 Further learning

The core and suggested reading can be found on the [reading list](#) as usual, and there's a nice [Quantitude podcast](#) if you want to consolidate your understanding. If you want to gain more hands-on practice in R, I recommend starting with the introductory DataCamp course on [Fundamentals of Bayesian Data Analysis in R](#).

! Feedback!

These practical sessions are new this year. Please take a moment to rate the difficulty and length of these practical activities via [this survey](#), and let us know of any issues you encountered or aspects you didn't understand (positive feedback is also welcome!). You will need to be logged in with your university account to respond, but your submission will be anonymous.

For more general questions about the practical, please post them on the [VLE Discussion Board](#).